



Playwright

Testes E2E e API no browser

Instalação, E2E, API enterprise, POM e padrões de migração

Automação • TypeScript • testflow-playwright

O que é o Playwright?

Framework de automação da **Microsoft** para testes end-to-end e API — com suporte nativo a Chromium, Firefox e WebKit.

- Escrito em **TypeScript** / JavaScript
- API moderna com **auto-wait** — menos flakiness
- Fixtures `page`, `request`, `context` por teste
- UI Mode, trace viewer, screenshots e relatório HTML
- Paralelismo nativo e isolamento entre workers

Por que usar Playwright?



Velocidade & paralelismo

Workers paralelos e browser contexts isolados por teste.



Locators resilientes

`getByTestId`, `role`, `text` — recomendados pela doc oficial.



Debug rico

UI Mode, headed, Inspector e trace on retry.



E2E + API unificados

`page` para UI e `request` para contratos REST.

Pré-requisitos

Ambiente

- **Node.js** 20+
- **npm** (ou pnpm / yarn)
- App TestFlow na porta **5050**
- Editor: VS Code / Cursor (extensão Playwright opcional)

```
$ node --version
v20.x.x

$ docker run -p 5050:5050 gaschool/testflow:latest
# app disponível em localhost:5050
```

Instalação — testflow-playwright

① Clone o repositório

```
git clone .../testflow-playwright && cd testflow-playwright
```

② Instale dependências

```
npm install
```

③ Instale o browser Chromium

```
npm run install:browsers
```

④ Execute os testes

```
npm test ou npm run test:ui
```

Proxy corporativo (Zscaler)? O script `install:browsers` contorna erro SSL. Veja também `NODE_EXTRA_CA_CERTS` no README.

Estrutura do projeto

```
testflow-playwright/
├─ fixtures/      # credentials.json
├─ pages/         # Page Object Model
├─ support/       # auth, helpers
├─ tests/
│   └─ api/       # contratos REST
│   └─ auth/
│   └─ smoke/
│   └─ dashboard/
│   └─ team/
│   └─ settings/
│       └─ components/
├─ playwright.config.ts
└─ package.json
```

- `pages/` — seletores, ações e assertions por tela
- `support/auth.ts` — login via API ou UI
- `tests/*.spec.ts` — specs por feature / suite
- `data-testid` — convenção de seletores estáveis

Configuração — playwright.config.ts

```
import { defineConfig } from '@playwright/test'

const BASE_URL = process.env.BASE_URL ?? 'http://localhost:5050'

export default defineConfig({
  testDir: './tests',
  timeout: 8000,
  use: {
    baseURL: BASE_URL,
    viewport: { width: 1280, height: 800 },
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
  projects: [
    { name: 'api', testMatch: /tests\/api\/.*\.spec\.ts/ },
    { name: 'smoke', testMatch: /tests\/smoke\/.*\.spec\.ts/ },
    { name: 'auth', testMatch: /tests\/auth\/.*\.spec\.ts/ },
    // dashboard, team, settings, components...
  ],
})
```

projects permitem rodar suites em paralelo no CI — mesmo padrão do testflow-cypress.

Seu primeiro teste

```
// tests/auth/login.spec.ts
import { test, expect } from '@playwright/test'
import { LoginPage } from '../../pages/LoginPage'
import { DEMO_EMAIL, DEMO_PASSWORD } from '../../support/helpers'

test.describe('Authentication', () => {
  test.beforeEach(async ({ page }) => {
    await new LoginPage(page).visit()
  })

  test('logs in via UI and redirects to dashboard', async ({ page }) => {
    const login = new LoginPage(page)
    await login.loginWith(DEMO_EMAIL, DEMO_PASSWORD)
    await login.shouldRedirectToDashboard()
  })
})
```

Convenção: `data-testid="login-email"` → `page.getByTestId('login-email')`

Locators & navegação

API Playwright usada na suíte TestFlow — seletores via `data-testid`

API	Uso no TestFlow
<code>page.goto(path)</code>	Abrir login, dashboard, team...
<code>page.getByTestId(id)</code>	Selector padrão — <code>login-email</code> , <code>page-team</code>
<code>page.getByRole(...)</code>	Botões, links acessíveis
<code>page.getByText(...)</code>	Labels visíveis, mensagens de erro
<code>locator.locator(...)</code>	<code>tbody tr</code> , células, colunas
<code>locator.nth(i)</code>	Linha n da tabela, item de activity
<code>page.waitForRequest(...)</code>	Spy POST login, GET users, PUT/PATCH row
<code>page.waitForResponse(...)</code>	Validar status após ação na UI
<code>page.evaluate(...)</code>	sessionStorage pós-login
<code>page.addInitScript(...)</code>	Injetar token antes do <code>goto</code>
<code>page.reload()</code>	Recarregar após <code>page.route</code> mock

Interações

Ação	Exemplo TestFlow
<code>.fill(text)</code>	Email, senha, busca na tabela
<code>.clear()</code>	Limpar busca antes de novo filtro
<code>.click()</code>	Submit, sidebar, paginação, modais
<code>.selectOption(value)</code>	Role filter, invite role, run suite
<code>.check()</code> / <code>.uncheck()</code>	Remember me, API toggle
<code>.press('Escape')</code>	Fechar modal de invite / new run
<code>.innerText()</code>	Extrair valor numérico de KPI
<code>.isVisible()</code>	Condicional — toast ou result msg
<code>.evaluate(el ⇒ el.click())</code>	Checkbox coberto por label custom
<code>page.keyboard.press(...)</code>	Atalhos de teclado na UI

Assertions — expect()

Assertion	Exemplo
<code>toBeVisible()</code>	Page root, modais, KPIs, toast
<code>not.toBeVisible()</code>	Modal fechado após submit
<code>toHaveURL(...)</code>	Redirect pós-login
<code>toContainText(...)</code>	Erros, greeting, row count label
<code>toHaveAttribute(...)</code>	type=password, placeholder
<code>toHaveCount(n)</code>	Linhas visíveis, colunas, activity items
<code>toHaveText(/regex/)</code>	Pass rate <code>^\d+%</code>
<code>toMatchObject({...})</code>	Payload do invite POST
<code>expect(res.status()).toBe(200)</code>	Contratos API
<code>expect(duration).toBeLessThan(ms)</code>	SLA de resposta API

Playwright **aguarda automaticamente** até a condição ser verdadeira — sem waits manuais na UI.

Helpers compartilhados

support/helpers.ts

<code>DEMO_EMAIL</code> · <code>DEMO_PASSWORD</code>	Credenciais demo via env
<code>validateSchema(obj, schema)</code>	Tipos da resposta REST
<code>getTableRows(page)</code>	Linhas de <code>users-table</code>
<code>getCell(page, rowId, field)</code>	Célula por id de linha

support/auth.ts

<code>loginViaApi(page, request)</code>	Auth programática + goto dashboard
<code>visitAuthenticated(page, request, path)</code>	Navegar já autenticado
<code>visitWithToken(page, path, token)</code>	Smoke — token do beforeAll
<code>loginViaUi(page)</code>	Auth via formulário
<code>authenticatedRequest(request, opts)</code>	HTTP com Bearer token
<code>getAuthToken(request)</code>	Token para specs paralelos

fixtures/

<code>credentials.json</code>	valid / invalid email e password
<code>team-member.json</code>	Payload do modal invite

Estrutura de specs

```
test.describe('Team', () => {
  test.beforeEach(async ({ page, request }) => {
    await visitAuthenticated(page, request, '/web/team.html')
    await expect(new TeamPage(page).pageRoot()).toBeVisible()
  })

  test.describe('Search', () => {
    test('filters rows by member name', async ({ page }) => {
      const team = new TeamPage(page)
      await team.search('Alice')
      await team.shouldHaveRowCount(1)
    })
  })

  test.describe('Invite modal', () => { /* ... */ })
})
```

describe aninhado

beforeEach setup

1 cenário / test()

beforeEach vs beforeAll

```
// beforeEach - estado fresco por teste
test.beforeEach(async ({ page, request }) => {
  await loginViaApi(page, request)
  await new DashboardPage(page).shouldBeLoaded()
})
```

```
// beforeAll - resposta API compartilhada
let res: APIResponse
let body: { users: User[] }

test.beforeAll(async ({ request }) => {
  const start = Date.now()
  res = await request.get('/api/users')
  duration = Date.now() - start
  body = await res.json()
})

test('returns status 200', () => {
  expect(res.status()).toBe(200)
})
```

`beforeAll` evita N chamadas idênticas; `beforeEach` isola estado de UI.

Data-driven tests

```
// tests/smoke/navigation.spec.ts
const PAGES = [
  { path: '/web/dashboard.html', testId: 'page-dashboard', title: 'Dashboard' },
  { path: '/web/team.html', testId: 'page-team', title: 'Team' },
  // settings, components, activity, wizard...
]

for (const { path, testId, title } of PAGES) {
  test(`${title} page loads without error`, async ({ page }) => {
    await visitWithToken(page, path, authToken)
    await expect(page.getByTestId(testId)).toBeVisible()
    await expect(page).toHaveTitle(new RegExp(title))
  })
}
```

Um loop gera N testes nomeados — smoke cobre todas as rotas com um único padrão.

test.step — relatório legível

```
// Padrão enterprise — steps no HTML report
test('GET /resources — Authorization header omitted', async ({ playwright }, testInfo) => {
  await test.step('New context without Authorization header', async () => {
    const ctx = await playwright.request.newContext({ baseURL })
    try {
      const response = await ctx.get('/api/resources')
      await attachHttpExchangeReport(testInfo, {
        label: 'resources-no-auth',
        method: 'GET',
        url: '/api/resources',
        response,
      })
      expect(response.status()).toBe(403)
    } finally {
      await ctx.dispose()
    }
  })
})
```

Equivalente ao `cy.step` — útil em testes negativos de API com múltiplas fases.

API testing — request fixture

```
// tests/api/auth.api.spec.ts
const ENDPOINT = '/api/auth/login'
const VALID = { email: DEMO_EMAIL, password: DEMO_PASSWORD }

test.describe('Valid credentials', () => {
  test.beforeAll(async ({ request }) => {
    res = await request.post(ENDPOINT, { data: VALID })
    body = await res.json()
  })

  test('returns status 200', () => expect(res.status()).toBe(200))
  test('body has token as non-empty string', () => {
    expect(typeof body.token).toBe('string')
  })
  test('responds within 2000ms', () => expect(duration).toBeLessThan(2000))
})
```

validateSchema — contrato REST

```
// support/helpers.ts
export function validateSchema(obj, schema: Schema) {
  for (const [key, type] of Object.entries(schema)) {
    expect(obj).toHaveProperty(key)
    expect(typeof obj[key]).toBe(type)
  }
}

// tests/api/users.api.spec.ts
for (const user of body.users) {
  validateSchema(user, { name: 'string', email: 'string', role: 'string' })
}
```

GET /api/users

GET /health

GET /api/errors/404

GET /api/errors/422

page.route — spy & mock

```
// Spy - payload e status do login API
const loginCall = page.waitForRequest(
  (req) => req.url().includes('/api/auth/login') && req.method() === 'POST',
)
await login.toggleUseApi()
await login.loginWith(DEMO_EMAIL, DEMO_PASSWORD)
const req = await loginCall
expect(req.postDataJSON().email).toBe(DEMO_EMAIL)
expect((await req.response()).status()).toBe(200)

// Mock - lista vazia de users
await page.route(/\/api\/users/, (route) =>
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ users: [] }),
  }),
)
await page.reload()
```

Intercept na UI — Team & Invite

```
// Spy POST invite
const inviteRequest = page.waitForRequest(
  (req) => req.method() === 'POST' && req.url().includes('/api/'),
).catch(() => null)

await team.openInviteModal()
await team.fillInviteForm(teamMember.new)
await team.submitInvite()

const req = await inviteRequest
if (req) {
  expect(req.postDataJSON().email).toBe(teamMember.new.email)
}

// Spy PUT ou PATCH inline edit
const writeReq = Promise.race([
  page.waitForRequest((r) => r.method() === 'PUT' && r.url().includes('/api/')),
  page.waitForRequest((r) => r.method() === 'PATCH' && r.url().includes('/api/')),
])
```

Auth helpers — setup rápido

```
// support/auth.ts
async function injectAuth(page, session) {
  await page.addInitScript((auth) => {
    sessionStorage.setItem('sandbox-auth', JSON.stringify(auth))
    sessionStorage.setItem('sandbox-token', auth.token)
  }, session)
}

export async function loginViaApi(page, request) {
  const session = await fetchAuthToken(request)
  await injectAuth(page, session)
  await page.goto('/web/dashboard.html')
  await page.getByTestId('page-dashboard').waitFor()
}
```

Usado em dashboard, team, settings — specs focam na feature, não no login.

Fixtures customizadas

Substitui comandos globais — injeta token e headers em todo `request`

```
// fixtures.ts - padrão enterprise
export const test = base.extend<{
  authToken: string
  userAccessToken: string
}>({
  authToken: async ({}, use) => {
    const token = await getOrRefreshSystemToken()
    await use(token)
  },
  request: async ({ request }, use) => {
    await use(wrapRequestWithDefaultHeaders(request, DEFAULT_HEADERS))
  },
})

test.afterEach(async ({}, testInfo) => {
  if (testInfo.status === 'failed') {
    await testInfo.attach('failure-details', { body: formatError(testInfo) })
  }
})
```

EXPECT — status codes tipados

```
// helpers/apiContract.helpers.ts
export const EXPECT = {
  happy: 200,
  noAuth: 403,
  invalidBearer: 401,
  notFound: 404,
  validationError: 422,
} as const

export const ATTACHMENT_LABELS = {
  happyPath: 'get-resources-200',
  noAuth: 'get-resources-no-auth',
} as const

test(`[${EXPECT.noAuth}] GET without Authorization`, async ({ request }) => {
  const res = await request.get('/api/resources')
  expect(res.status()).toBe(EXPECT.noAuth)
})
```

Route modules + exchange helpers

```
// core/routes.ts
export const AUTH_LOGIN_PATH = '/api/auth/login'
export function authLoginUrl(base: string) {
  return `${base}${AUTH_LOGIN_PATH}`
}

// helpers/authExchange.helpers.ts
export async function postLoginExchange(
  request, testInfo, label, body
) {
  const response = await request.post(AUTH_LOGIN_PATH, { data: body })
  return attachHttpExchangeReport(testInfo, {
    label, method: 'POST', url: AUTH_LOGIN_PATH, response,
  })
}
```

- URLs centralizadas — sem strings espalhadas
- Exchange helper = request + attach report
- Assertion helper valida shape do JSON
- Path aliases: `@pw/core/*`, `@pw/helpers/*`

attachHttpExchangeReport

```
// helpers/apiExchange.ts - debug no HTML report
await attachHttpExchangeReport(testInfo, {
  label: 'login-200',
  method: 'POST',
  url: '/api/auth/login',
  requestHeaders: { Authorization: 'Bearer ...' }, // mascarado
  response,
})
// Anexa: status, headers, body JSON, curl reproduzível
```

testInfo.attach

curl replay

mask Authorization

mask passwords

Tags, grep & modo serial

```
test.describe('Happy path suite @needs-browser', () => {
  test.describe.configure({ mode: 'serial', timeout: 150_000 })

  test.beforeAll(async () => { /* setup MFA / OAuth once */ })
  test.afterAll(async () => { /* teardown */ })

  for (const scope of SCOPES) {
    test(`initiate + confirm [${scope}]`, async ({ request }) => { /* ... */ })
  }
})

test.describe('Negative testing', { tag: '@negative-testing' }, () => { /* ... */ })
```

```
# CI — jobs separados por tag
npx playwright test --grep-invert '@needs-browser' # API only, Alpine
npx playwright test --grep '@needs-browser' # OAuth browser flow
npx playwright test --grep '@negative-testing'
```

globalSetup — cache de sessão

```
// playwright.config.ts
export default defineConfig({
  globalSetup: require.resolve('./globalSetup'),
})

// globalSetup.ts — roda uma vez antes de todos os workers
// 1. Reutiliza token em .playwright/token-cache.json
// 2. Tenta refresh_token grant
// 3. Fallback: browser OAuth PKCE (Identity Provider)
// 4. Limpa estado MFA entre runs
```

Equivalente ao `cy.session` — login pesado uma vez, reutilizado em N specs.

API enterprise — visão geral

Suíte API-only: sem browser, sem UI — foco em contratos REST, OAuth e sync entre serviços.

Legado Cypress

Service Objects + `cy.request` + localStorage token

Playwright atual

`request` fixture + helpers + JWT cache

Organização

`api/gets/` · `api/patch/` · fixtures JSON

CI

`PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1`

```
profile-api-playwright/
├─ fixtures.ts      # authToken + testUser override
├─ auth.ts          # OAuth + JWT cache
├─ env.ts           # sit / vat / dual gateway
├─ headers.ts       # context headers obrigatórios
├─ test-users.json  # usuários nomeados por community
├─ api/
│   └─ gets/lookups/ # golden JSON contracts
│       └─ patch/    # data-driven PATCH suites
```

Service Objects — Cypress legado

Equivalente a POM, mas para API — encapsula URLs, auth e métodos HTTP.

```
// cypress/models/ProfileApiClient.ts
export class ProfileApiClient {
  flushProfile() {
    return cy.request({
      method: 'PUT',
      url: this.url.flushProfile,
      headers: { Authorization: `Bearer ${localStorage.getItem('access_token')}` },
    }).its('status').should('eq', 204)
  }

  getCountries() {
    return cy.request({ method: 'GET', url: this.url.countries, headers: { /* ... */ } })
  }

  patchCustomerName(body) {
    return cy.request({ method: 'PATCH', url: this.url.name, body, headers: { /* ... */ } })
  }
}

// spec — alias no before()
cy.wrap(new ProfileApiClient()).as('profileApi')
cy.wrap(new LifecyclePlatformClient({ userId })).as('lifecycleApi')
```

Auth cache — Cypress → Playwright

```
// Cypress - localStorage plugin
before() => {
  precondition.loginByApi({ username, password })
  cy.saveLocalStorage()
}
beforeEach() => cy.restoreLocalStorage()
```

```
// Playwright - JWT parse + Map cache
export async function getOrRefreshToken(user) {
  const cached = tokenCache.get(user.username)
  if (cached && isTokenValid(cached)) return cached
  const token = await fetchTokenFromOAuth(user)
  tokenCache.set(user.username, token)
  return token
}

// bypass total via env
if (process.env.AUTH_TOKEN) return process.env.AUTH_TOKEN
```

test.use — override de usuário por suite

```
// fixtures.ts
export const test = base.extend<{
  authToken: string
  testUser: TestUserCredentials | null
}>({
  testUser: [null, { option: true }],
  authToken: async ({ testUser }, use) => {
    const token = testUser
      ? await getOrRefreshTokenForUser(testUser)
      : await getOrRefreshBusinessToken()
    await use(token)
  },
})

// lending-partner.spec.ts — community diferente
import { getTestUser } from '../test-users'
test.use({ testUser: getTestUser('lendingPartner') })

test.describe('PATCH profile @patch @lending', () => { /* ... */ })
```

test-users.json

tokenEndpoint override

communityId claim

cache por usuário

Headers de contexto obrigatórios

```
// headers.ts — espelha Postman pre-request script
export function buildApiHeaders(options: {
  profileContext: 'KYC' | 'OAA'
  lineOfBusiness?: 'Brokerage' | 'ConsumerLending'
}) {
  return {
    'X-Profile-Context': options.profileContext,
    'Correlation-Id': uuid(),
    'Session-Id': uuid(),
    'Client-Id': '192.168.1.1',
    LineOfBusiness: options.lineOfBusiness ?? 'Brokerage',
    SourceLob: 'playwright-automation',
  }
}

// uso no spec
const headers = {
  Authorization: `Bearer ${authToken}`,
  ...buildApiHeaders({ profileContext: 'KYC', lineOfBusiness: 'ConsumerLending' }),
}

await request.patch('/v1/customers/me/profile/employment/employer', { headers, data })
```

Golden fixtures — contrato sem JSON Schema

```
// api/gets/lookups/helpers.ts
export async function fetchLookup(request, authToken, path, testInfo) {
  await testInfo.attach('request-headers', { body: JSON.stringify(headers) })
  const response = await request.get(path, { headers: { Authorization: `Bearer ${authToken}` } })
  await testInfo.attach('response-body', { body: await response.text() })
  return { response, body: await response.json() }
}

export function withoutId(arr) {
  return arr.map(({ id, ...rest }) => rest) // ignora id dinâmico
}

export function expectSameMembers(expected, received) {
  // compara arrays ignorando ordem — diff legível no report
}
```

```
// spec
const { body } = await fetchLookup(request, authToken, '/v1/lookups/countries', testInfo)
expect(response.status()).toBe(200)
expect(withoutId(body)).toEqual(expectedCountriesFixture)
```

Data-driven — runPatchTests()

```
// employment-status-shared.ts
export function runPatchTests(context, { payloads, tag }) {
  for (const [label, payload] of Object.entries(payloads)) {
    test.describe(label, () => {
      test(`204 - PATCH accepted, GET profile matches`, async ({ request, authToken }) => {
        const headers = {
          Authorization: `Bearer ${authToken}`,
          ...buildApiHeaders({ profileContext: context }),
        }
        const patchRes = await request.patch(PATCH_PATH, { headers, data: payload })
        expect(patchRes.status()).toBe(204)

        const profile = await pollGetUntilMatch(request, GET_PATH, headers, payload)
        expect(profile).toMatchObject(normalizePayload(payload))
      })
    })
  }
}

// spec — gera dezenas de testes a partir de um mapa de payloads
runPatchTests('KYC', { payloads: EMPLOYED_PAYLOADS, tag: 'employmentCode' })
```

PATCH → poll GET — eventual consistency

```
async function pollGetUntilMatch(request, path, headers, expected, deadlineMs = 10_000) {
  const start = Date.now()
  while (Date.now() - start < deadlineMs) {
    const res = await request.get(path, { headers })
    if (res.status() === 200) {
      const body = normalizeProfile(await res.json())
      if (matchesExpected(body, expected)) return body
    }
    await new Promise((r) => setTimeout(r, 500))
  }
  throw new Error('GET did not match PATCH payload within deadline')
}
```

Backend assíncrono — não assert imediato após PATCH; poll com intervalo de 500ms.

Verificação dual write + read

```
// Cypress legado - valida sync entre dois serviços
profileApi.patchCustomerName(nameBody).should(r => expect(r.status).to.eq(204))
profileApi.getBasicProfile().should(r => {
  expect(r.body.pendingProfile.customerName).to.deep.include(nameBody)
})
lifecycleApi.getBasicProfile().should(r => {
  expect(r.body.pendingProfile).to.deep.include(nameBody)
})
```

PATCH no Profile API + GET no Lifecycle Platform confirma propagação do estado pendente.

Diff + curl no report de falha

```
function diffForObject(expected, received, path = '') {
  const lines = []
  for (const [key, exp] of Object.entries(expected)) {
    const recv = received[key]
    if (JSON.stringify(exp) !== JSON.stringify(recv)) {
      lines.push(`- ${path}.${key}: expected ${JSON.stringify(exp)}, got ${JSON.stringify(recv)}`)
    }
  }
  return lines
}

// em falha de GET pós-PATCH
await testInfo.attach('comparison-diff', {
  body: buildValidationReport(label, expected, received),
  contentType: 'text/markdown',
})
await testInfo.attach('curl-replay', { body: buildCurl('PATCH', url, headers, body) })
```

Postman → Playwright migration

- `postman/AUTH.md` documenta fluxo OAuth
- Pre-request script → `buildApiHeaders()`
- Tests tab assertions → `expect()` + golden fixtures
- Collection env vars → `.env` + GitLab CI/CD variables
- JWT validation → `isTokenValid(token, claims)`

```
export function isTokenValid(token, conditions) {  
  const payload = parseJwt(token)  
  if (payload.typ !== 'at+jwt') return false  
  if (payload.exp * 1000 < Date.now() + 60_000) return false  
  if (conditions.preferred_username)  
    return payload.preferred_username === conditions.preferred_username  
  return true  
}
```

CI — env diagnostics & skip condicional

```
// api/ci/env-diagnostics.spec.ts
test('required CI variables are set', async () => {
  expect(process.env.CLIENT_ID, 'CLIENT_ID').toBeTruthy()
  expect(process.env.CLIENT_SECRET, 'CLIENT_SECRET').toBeTruthy()
  expect(process.env.API_BASE_URL, 'API_BASE_URL').toBeTruthy()
})

// skip quando credencial ausente
test.beforeAll(async () => {
  test.skip(!process.env.OAUTH_CLIENT_ID, 'OAuth client not configured in CI')
})
```

```
# .gitlab-ci.yml
playwright-api-tests:
  variables:
    PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD: "1"
  script:
    - npx playwright test
  artifacts:
  reports:
    junit: test-results/junit.xml
```

Page Object Model (POM)

Encapsula seletores, ações e assertions — o spec descreve o *cenário*, não o DOM. Para API-only: use **Service Objects** ou helpers (ver seção anterior).

Spec (.spec.ts)

- Given / When / Then
- Sem seletores espalhados
- Um cenário por `test()`



Page Object

- `getByTestId` locators
- Actions (`loginWith`)
- Assertions (`shouldRedirect...`)

Anatomy — LoginPage.ts

```
export class LoginPage {
  constructor(private readonly page: Page) {}

  emailInput() { return this.page.getByTestId('login-email') }
  submitBtn() { return this.page.getByTestId('login-submit') }

  async visit() {
    await this.page.goto('/web/login.html')
    return this
  }

  async loginWith(email: string, password: string) {
    await this.emailInput().fill(email)
    await this.passwordInput().fill(password)
    await this.submitBtn().click()
    return this
  }

  async shouldRedirectToDashboard() {
    await expect(this.page.getByTestId('page-dashboard')).toBeVisible()
    await expect(this.page).toHaveURL(/\/web\/dashboard\.html/)
  }
}
```

TeamPage — locators & ações

Locators

<code>tableRows()</code>	<code>users-table tbody tr</code>
<code>nameCell(id)</code>	<code>cell-name-{id}</code>
<code>inviteModal()</code>	Modal de convite

Actions

<code>search(term)</code>	Filtrar tabela
<code>filterByRole(role)</code>	Select de role
<code>goToNextPage()</code>	Paginação
<code>openInviteModal()</code>	Abrir + assert visible
<code>startEdit(id)</code>	Inline edit row

Assertions

<code>shouldHaveRowCount(n)</code>	Contagem de linhas
<code>shouldShowInviteError(text)</code>	Validação do modal

Page Objects do TestFlow

LoginPage.ts auth, validação, API toggle

DashboardPage.ts KPIs, activity, new run modal

TeamPage.ts tabela, invite, inline edit

SettingsPage.ts forms, toggles, save

```
// dashboard.spec.ts - POM + auth helper
test.beforeEach(async ({ page, request }) => {
  await loginViaApi(page, request)
  await new DashboardPage(page).shouldBeLoaded()
})

test('shows a percentage in the pass rate card', async ({ page }) => {
  await expect(new DashboardPage(page).kpiValue('passrate'))
    .toHaveText(/^\d+(\.\d+)?%$/ )
})
```

Cobertura da suíte

Project	Spec	O que cobre
smoke	navigation.spec.ts	Páginas, sidebar, health API, logout
auth	login.spec.ts	Login UI, sessionStorage, erros
dashboard	dashboard.spec.ts	KPIs, activity, modal new run
team	team.spec.ts	Busca, filtros, paginação, invite
settings	settings.spec.ts	Formulário de configurações
components	components.spec.ts	Biblioteca de UI + ComponentsPage
widgets	widgets/widgets.spec.ts	Classic Widgets — upload, jobs, selects
wizard	wizard/wizard.spec.ts	Multi-step, validação, review
activity	activity/activity.spec.ts	API mocks, counter, pipeline
advanced	advanced/advanced.spec.ts	Shadow DOM, iframe, mobile
states	states/states.spec.ts	Skeleton, empty, error states
layout	layout/shell.spec.ts	Nav, notificações, theme, mobile
a11y	a11y/a11y.spec.ts	axe-core em páginas-chave
api	api/*.spec.ts	REST, rules/PATCH, env diagnostics

~288 testes · 18 projects · multi-browser + visual + runPatchTests()

Como executar



UI Mode — explorar no browser

```
npm run test:ui  
npm run test:auth -- --ui
```



Headed — ver execução

```
npm run test:headed  
npm run test:team -- --headed
```

```
npm test           # todos os projects  
npm run test:smoke # só smoke  
npm run test:api   # só API  
npm run report     # relatório HTML
```

CI — GitHub Actions

Matrix paralela

17 projects + job @smoke — inclui Firefox, WebKit e visual

Service container

`gaschoo1/testflow:latest` na porta 5050

Artefatos

HTML report + screenshots em falha

Retries

2 retries no CI · trace on first retry

Workflow: `.github/workflows/playwright.yml`

Boas práticas

- **Page Objects** em `pages/` — locators, actions e assertions isolados
- `getByTestId` + `data-testid` — mesma convenção do testflow-cypress
- `loginViaApi` no `beforeEach` — specs de feature sem repetir login UI
- `beforeAll` + resposta compartilhada — contratos API sem N requests idênticos
- `validateSchema` — tipos da resposta REST de forma declarativa
- `page.route` + `waitForRequest` — spy payload e mock de erro
- Fixtures JSON — dados determinísticos (`credentials` , `team-member`)
- Um cenário por `test()` — falhas fáceis de diagnosticar no report
- `projects` no config — smoke, auth, api em paralelo no CI
- `test.step` + `attachHttpExchangeReport` — debug de API no HTML report
- `EXPECT` constants — status codes nomeados, não magic numbers
- Tags `@needs-browser` / `@negative-testing` — filtrar jobs no CI
- `globalSetup` + cache de token — setup pesado uma vez por run
- **Service Objects** (API) — encapsular `cy.request` / `request` por domínio
- `test.use({ testUser })` — override de credencial por suite / community

Próximos passos

Você já sabe instalar, configurar e escrever testes E2E e API com Playwright no TestFlow.



Documentação
playwright.dev



UI Mode
Debug interativo



TestFlow
App sob teste

← → slides | ↓ ↑ sub-slides | F tela cheia | ESC overview